

2. Introduction

Chapter I

Introduction

Dr.Eman Omar

Outline

- What is a distributed database system?
- Promises of DDBSs
- Complicating Factors
- Problem Areas



Traditional File System

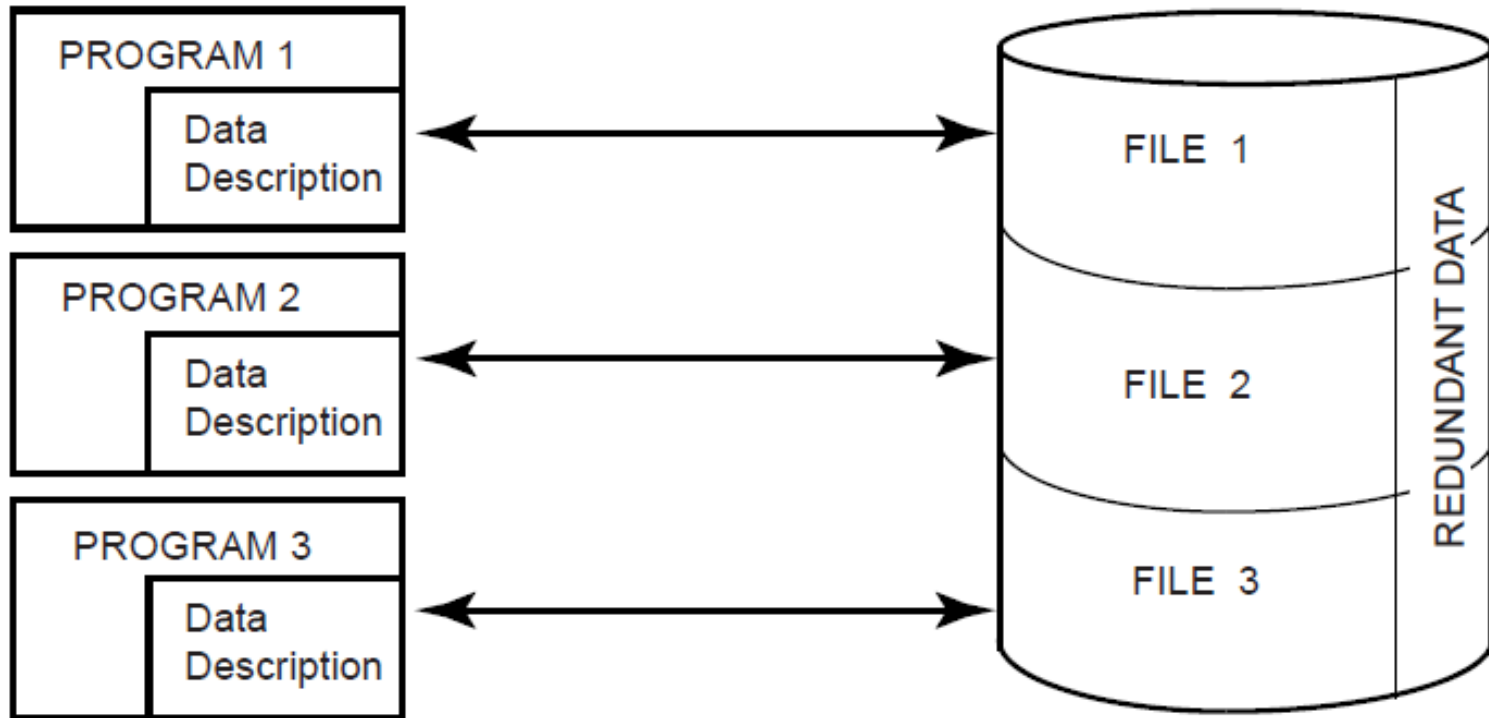
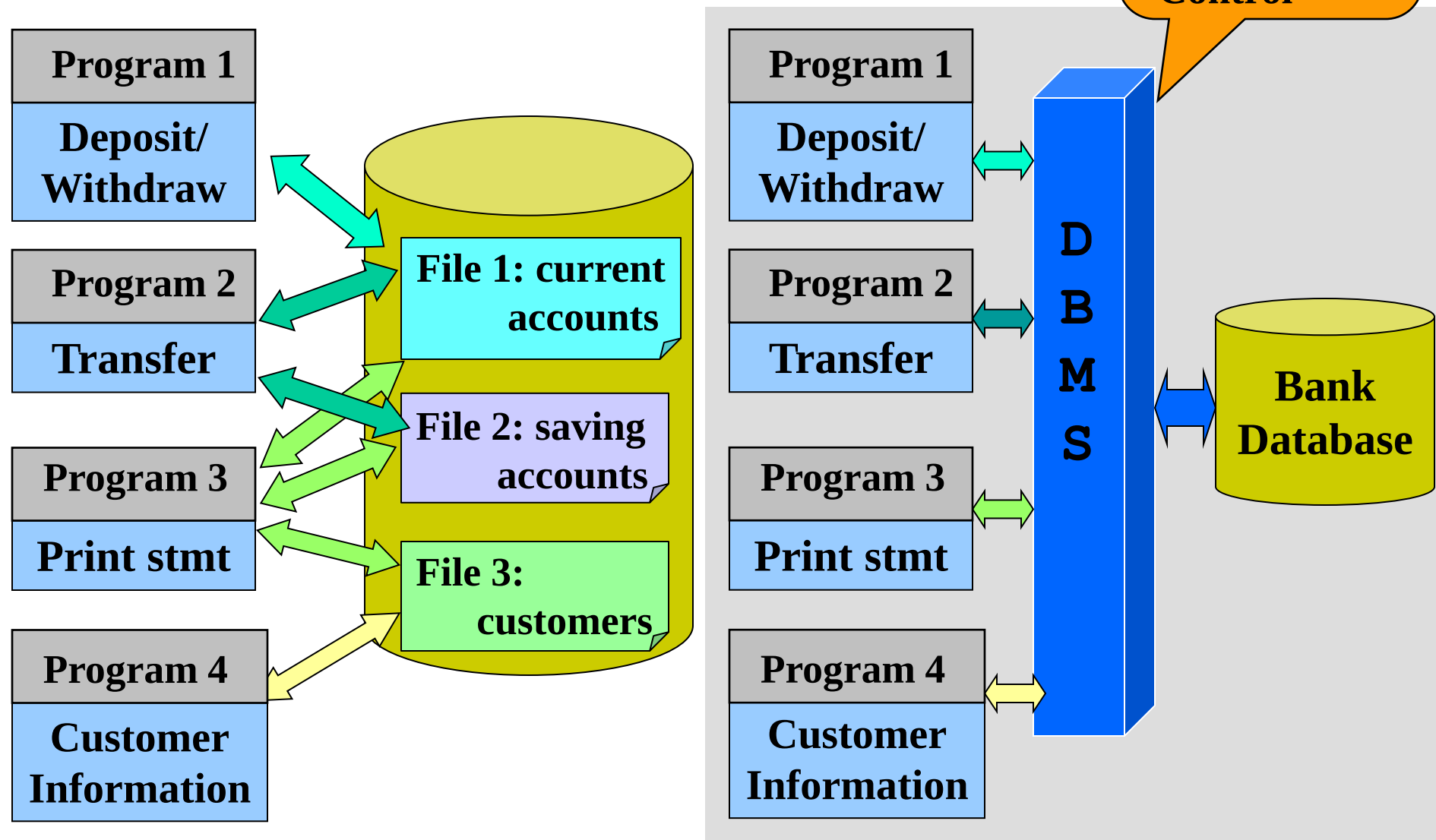


Fig. 1.1 Traditional File Processing

From File System to DBMS

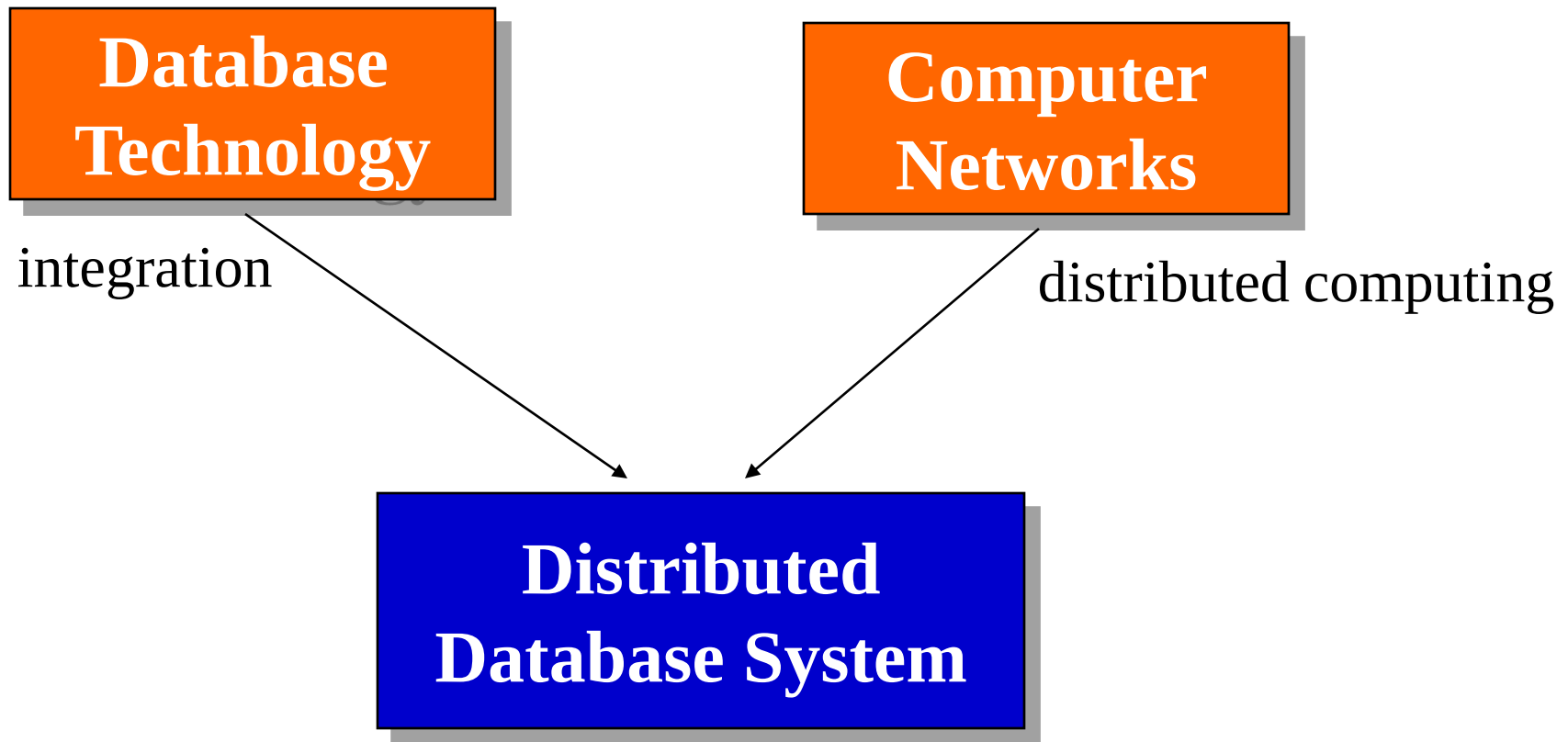
- Description
- Store
- Manipulation
- Control



What is a distributed DB?

- A **distributed database** is a collection of multiple interconnected databases, which are spread physically across various locations that communicate via a computer network.

Distributed Database System Technology



- The key is integration, not centralization
- Distributed database technology attempts to achieve integration without centralization.

DDBS = Database + Network

- Distributed database system technology is the union of what appear to be diametrically opposed approaches to data processing
 - ◆ DDBS = database + network
 - ◆ **Database** integrates operational data of an enterprise to provide a centralized and controlled access to that data.
 - ◆ **Computer network** promotes a work mode that goes against all centralization efforts and facilitates distributed computing.

Features of DDBS

1. Databases in the collection are logically interrelated with each other. Often they represent a single logical database.
2. Data is physically stored across multiple sites. Data in each site can be managed by a DBMS independent of the other sites.
3. The processors in the sites are connected via a network. They do not have any multiprocessor configuration.
4. A distributed database is not a loosely connected file system.
5. A distributed database incorporates transaction processing, but it is not synonymous with a transaction processing system.

Distributed Computing

- A distributed computing system consists of
 - ◆ a number of autonomous processing elements (**not necessarily homogeneous**), which
 - are interconnected by a computer network;
 - cooperate in performing their assigned tasks.

- What is distributed?
 - ◆ Function
 - ◆ Data
 - ◆ Control
 - ◆ Processing Logic

All these are necessary and important for distributed database technology.

Distributed Database Management System (DDMS)

- DDBMS is a centralized software system that manages a distributed database in a manner as if it were all stored in a single location.
- **DDBMS Features**
 1. It is used to create, retrieve, update and delete distributed databases.
 2. It synchronizes the database periodically and provides access mechanisms by the virtue of which the distribution becomes transparent to the users.
 3. It ensures that the data modified at any site is universally updated.
 4. It is used in application areas where large volumes of data are processed and accessed by numerous users simultaneously.
 5. It is designed for heterogeneous database platforms.
 6. It maintains confidentiality and data integrity of the databases.

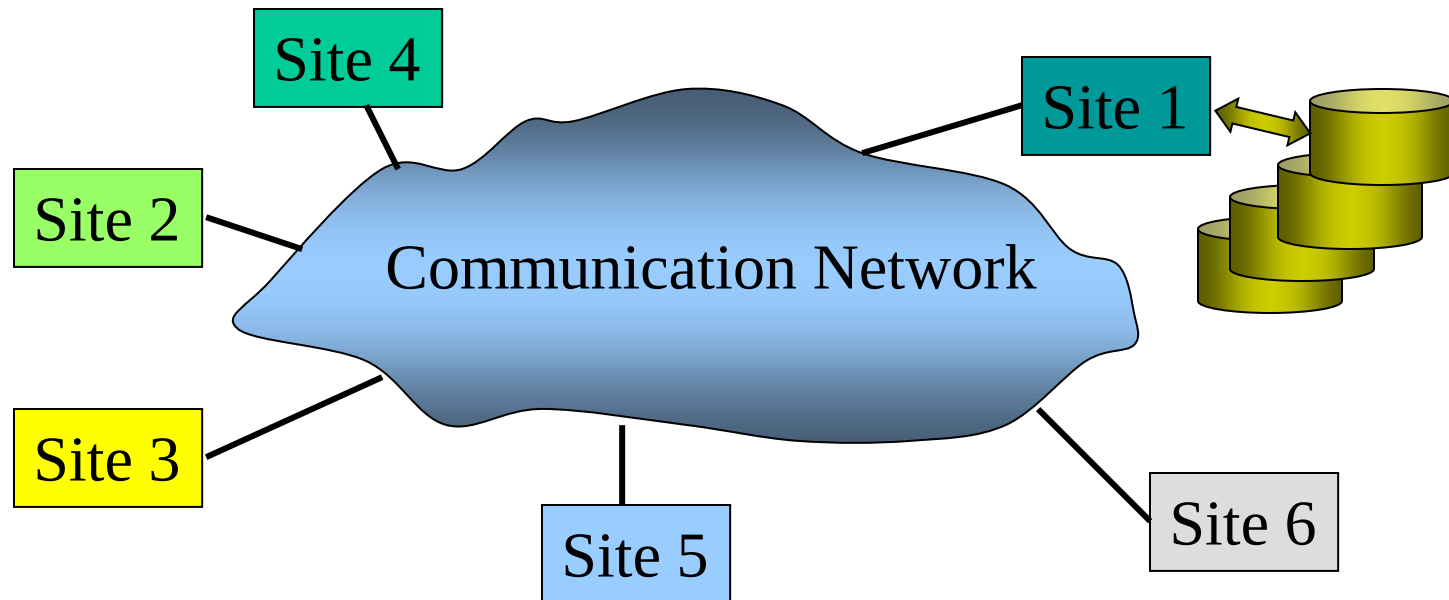
What is a Distributed Database System?

- A distributed database (DDB) is a collection of multiple, *logically interrelated* databases, distributed over a computer network
 - ◆ i.e., storing data on multiple computers (nodes) over the network
- A distributed database management system (DDBMS) is the software that
 - ◆ manages the DDB;
 - ◆ provides an access mechanism that makes this distribution *transparent* to the users.
- Distributed database system (DDBS):
 - ◆ **DDBS = DDB + DDBMS**

What is not a DDBS?

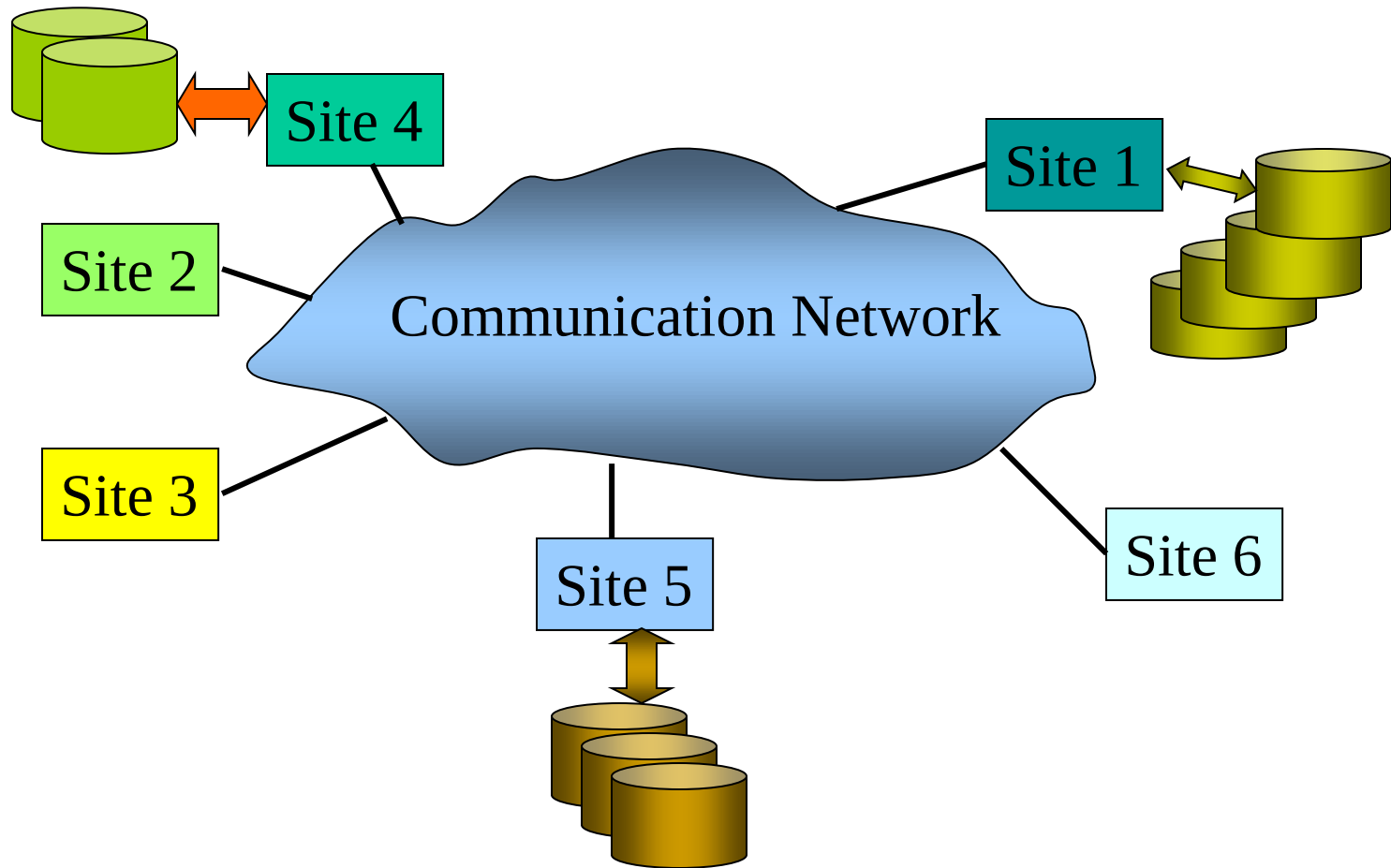
- A timesharing computing system
- A loosely or tightly coupled multiprocessor system
- A database system which resides at one of the nodes of a network of computers
 - ◆ This is a centralized database on a network node

Centralized DBMS on a Network



- Data resides only at one node.
- Database management is the same as in a centralized DBMS.
- Remote processing, single-server multiple-clients

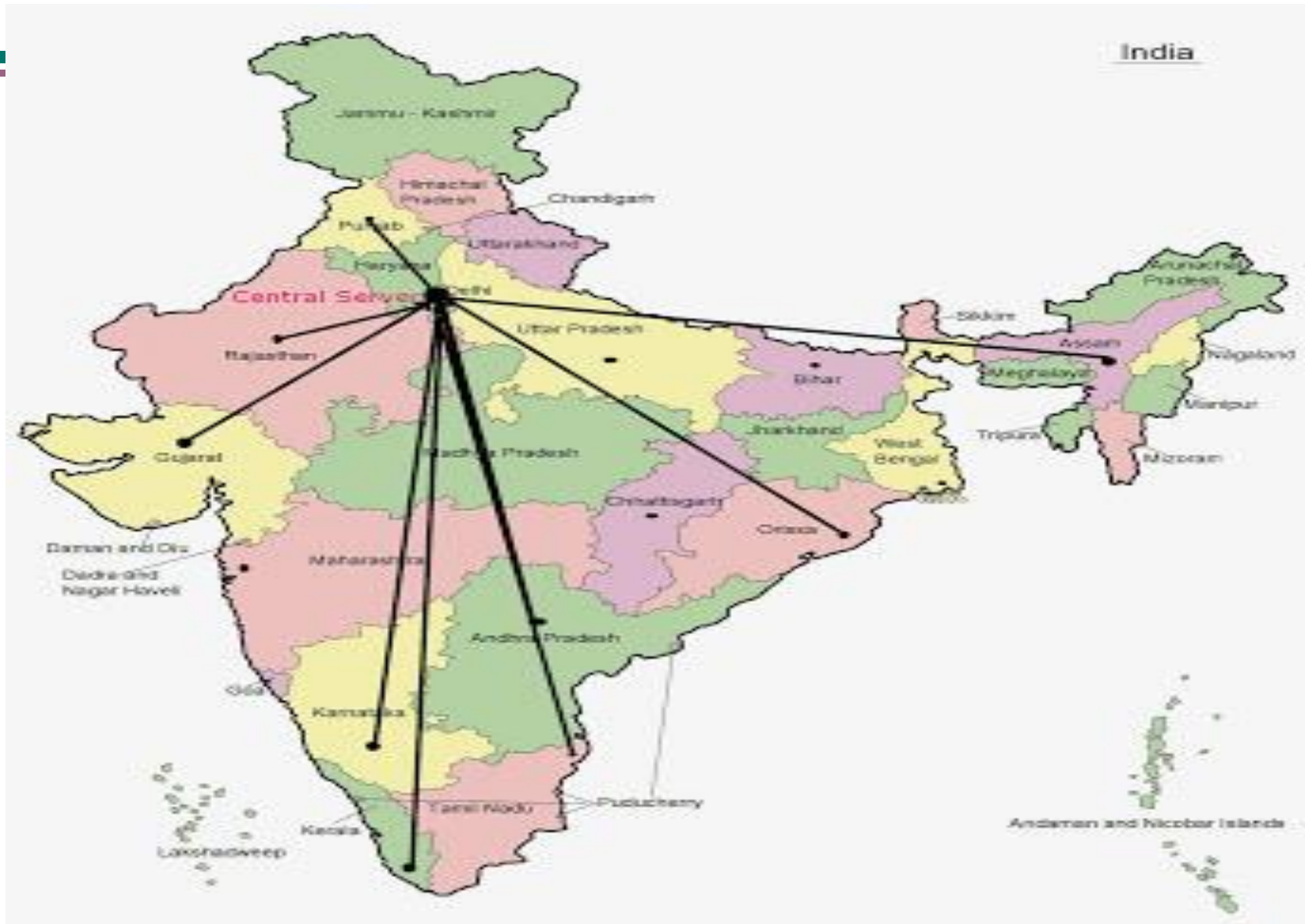
Distributed DBMS Environment



Why DDB?

- Let us consider **XYZ bank** which is headquartered in New Delhi. Also, assume that the bank maintains its server in its head office. Now, all the bank transactions done at all the branches of XYZ bank must reach the central server to access the data.
- For example, consider a customer who is trying to withdraw the money from his account through an ATM located in Chennai. **His withdrawal request must be sent to the central server, processed in central server**, and money will be disbursed in the ATM.
- The following image shows the Central Server approach for any database for any organization. The **requests** initiated are shown in thick lines.

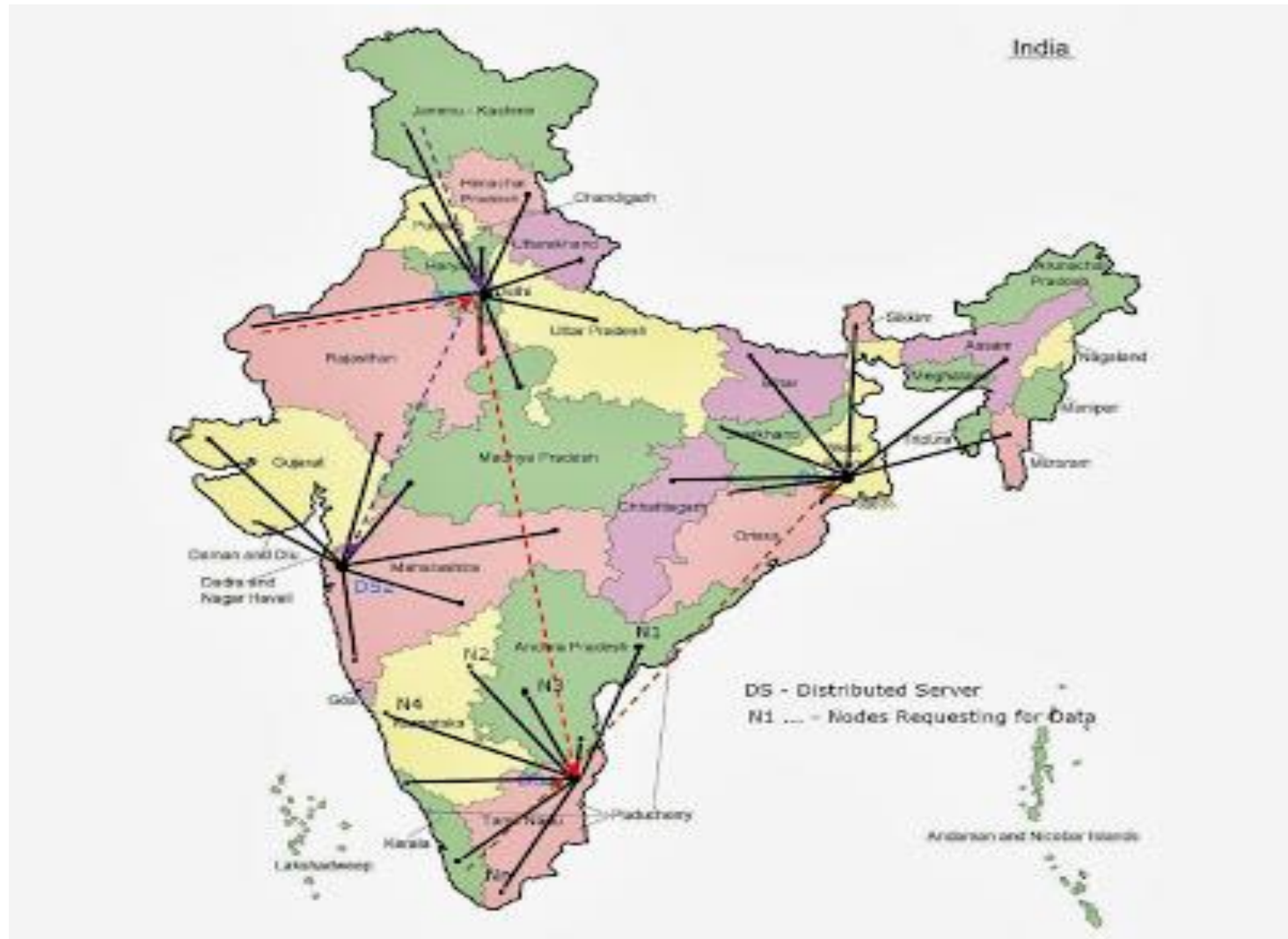
Centralized DB



DDB

- Now assume that, **XYZ bank** established several servers which are distributed throughout the country, say **10 different servers**.
- Now, any request generated from the ATM from any part of the country will be forwarded to the server available in that part of the country.
- For any reason, if the requested data is not available with the local server, the server searches for the actual location of the requested data and forwards the request to that server, and routes the answer to the initiator.
- The image shown below depicts the distributed server concept. It shows a set of DSs(Distributed Servers), a set of Nodes (not all are labeled), and a set of links which shows the request generated from node to the DS.
- The dashed line shows that the request generated by a node which is local to some other DS and the received DS forwarded to other DS where the intended data would be available.
- Here, the main advantage is consumption of network bandwidth is controlled, .i.e., network traffic reduced. Availability of the data and the server increased, as they are very close and accessible.

DDB



Applications of DDBMS

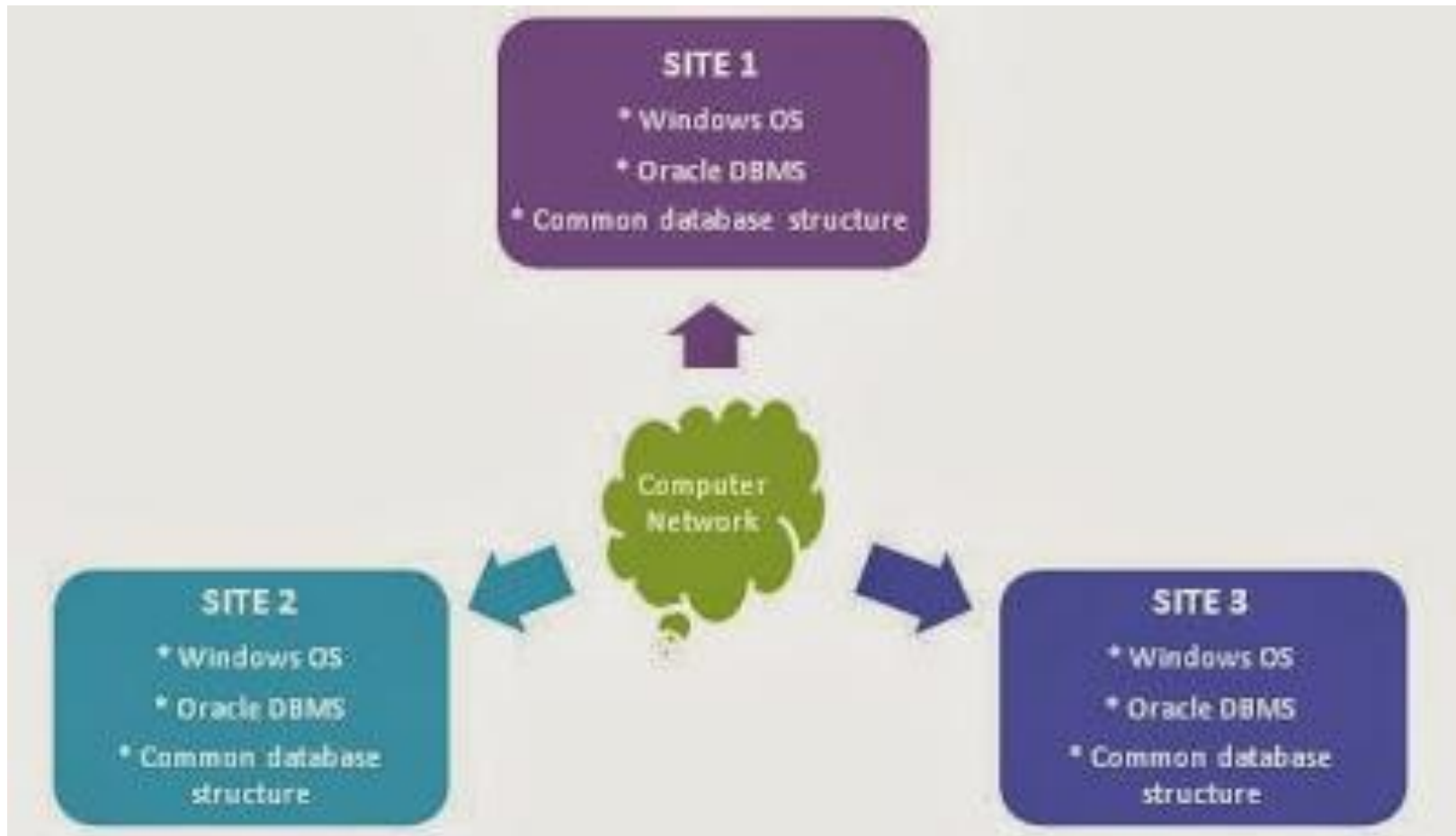
- Manufacturing – especially multi-plant manufacturing
- Military command and control
- Airlines
- Hotel chains
- Any organization which has a decentralized organization structure

Types of Distributed Databases

1. **Homogeneous Distributed Database**

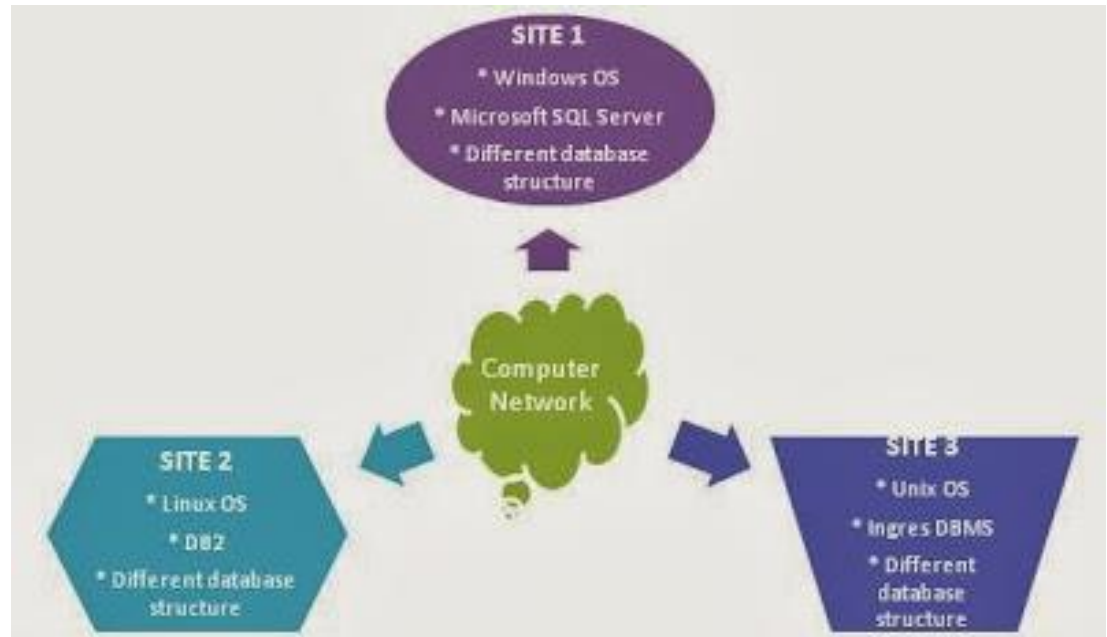
- ❖ Identical software are used in all the sites
 - ◆ Here, a site refers to a server which is part of the distributed database system
 - ◆ Software would mean OS, DBMS software, and even the structure of the database used
 - ◆ In some cases, identical hardware are also used
- All sites are well known: as they are similar in terms of DBMS software and hardware used
- Partial control over the data of other sites
 - ◆ As we know the structure of databases, software and hardware used in other sites. Hence the partial control over the data is possible
- Looks like a single central database system

Homogeneous Distributed Database



2. Heterogeneous Distributed Database

- Different sites use different database software
- The structure of databases reside in different sites may be different (because of data partitions)
- Co-operation between sites are limited. That is, it is not easy to alter the structure of the database or any other software used



Reasons for Data Distribution

- Several factors leading to the development of DDBS
 - ◆ Increased reliability and availability
 - ◆ Allowing data sharing while maintaining some measure of local control
 - ◆ Improved performance
 - ◆ Distributed nature of some database applications

Implicit Assumptions

- Data stored at a number of sites.
 - ◆ Each site has processing power
- Processors at different sites are interconnected by a computer network
- Distributed database is a database, not a collection of files
 - ◆ Data logically related as exhibited in the users' access patterns (e.g., relational data model)

Replication (in Database)

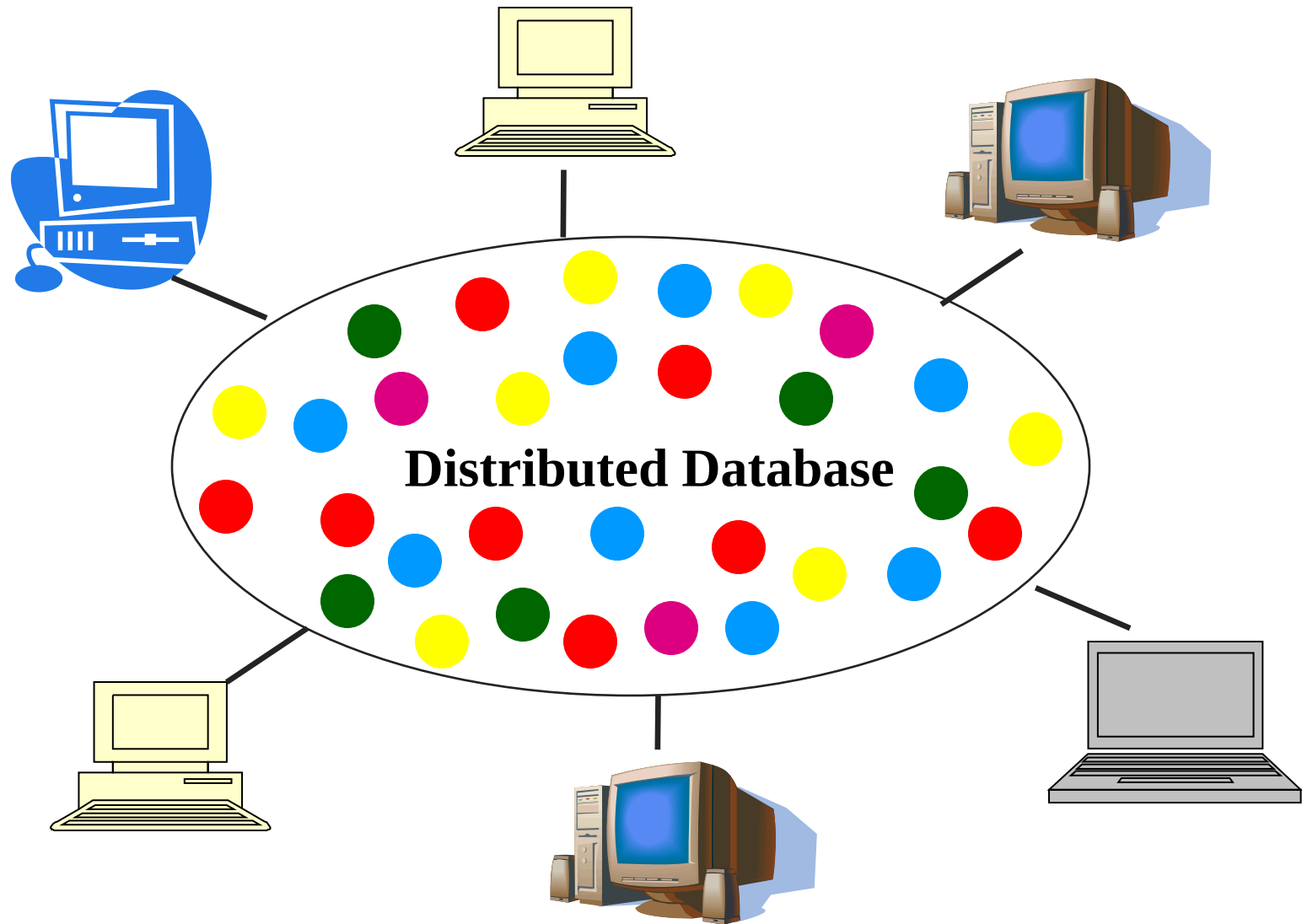
- Storing identical copies of same relations (tables) in different sites in a distributed database is called Replication.
- For example, a relation R is sent to several sites as R . At the end of replication in n different sites, the following will hold;
- $R \text{ in Site } 1 = R \text{ in Site } 2 = R \text{ in Site } 3 = \dots = R \text{ in Site } n$

Replication (in Database)

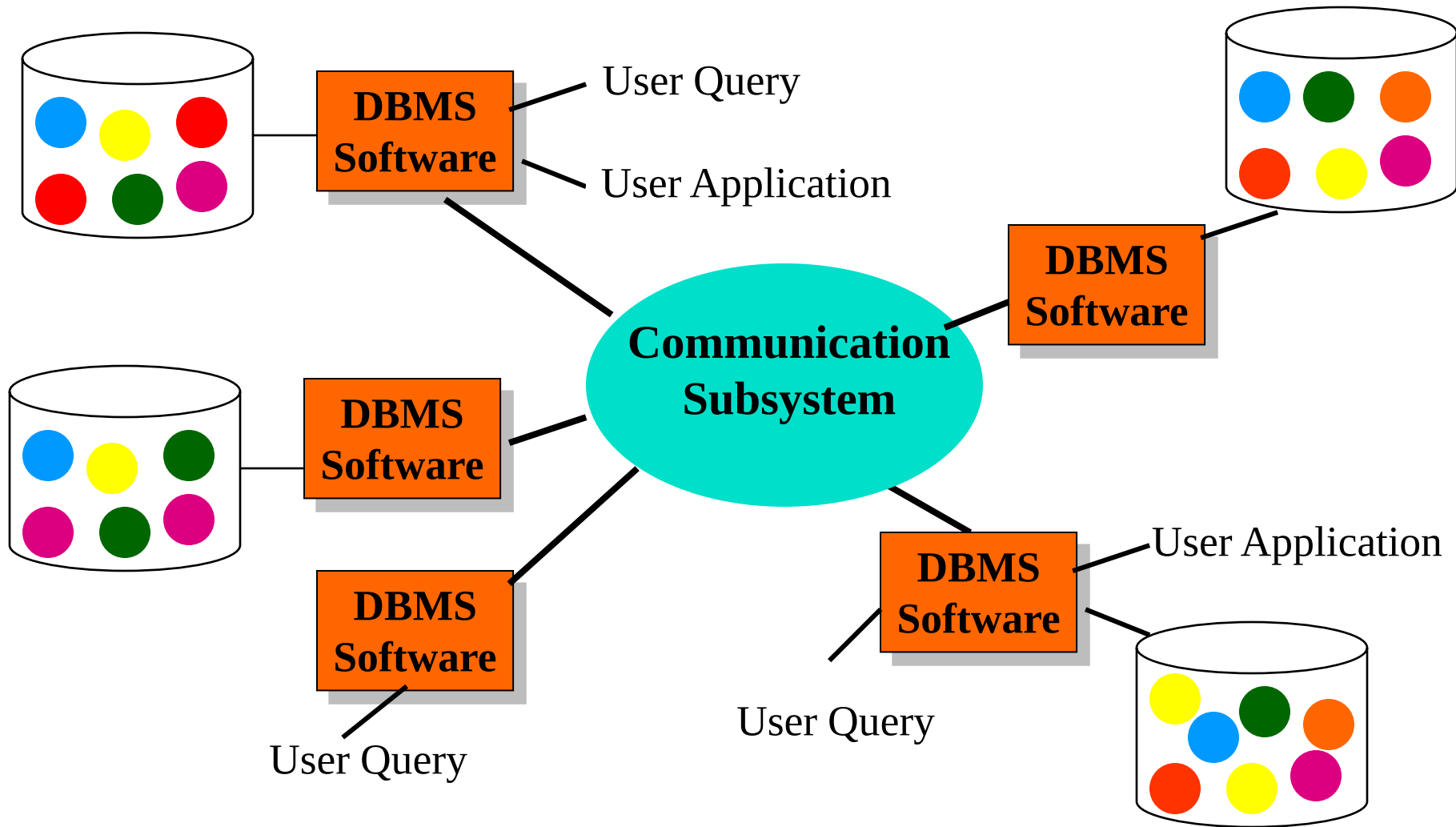
- Partitioning a relation (table) vertically or horizontally into several partitions and storing each partition in different site is called **Fragmentation** in distributed database.
- For example, a relation R is horizontally fragmented into n fragments. At the end of Horizontal fragmentation, every fragment is sent to n sites as follows;
- $R = R_1 \cup R_2 \cup R_3 \cup \dots \cup R_n$
- Or, a relation R is vertically fragmented into n fragments, and sent to n sites as follows;

$$R = R_1 \bowtie R_2 \bowtie R_3 \bowtie \dots \bowtie R_n$$

Distributed Database – User View



Distributed DBMS – Reality



Example Relations

EMP

ENO	ENAME	TITLE
E1	J. Doe	Elect. Eng.
E2	M. Smith	Syst. Anal.
E3	A. Lee	Mech. Eng.
E4	J. Miller	Programmer
E5	B. Casey	Syst. Anal.
E6	L. Chu	Elect. Eng.
E7	R. Davis	Mech. Eng.
E8	J. Jones	Syst. Anal.

ASG

ENO	PNO	RESP	DUR
E1	P1	Manager	12
E2	P1	Analyst	24
E2	P2	Analyst	6
E3	P3	Consultant	10
E3	P4	Programmer	48
E4	P2	Manager	18
E5	P2	Manager	24
E6	P4	Engineer	48
E7	P3	Engineer	36
E7	P5	Engineer	23
E8	P3	Manager	40

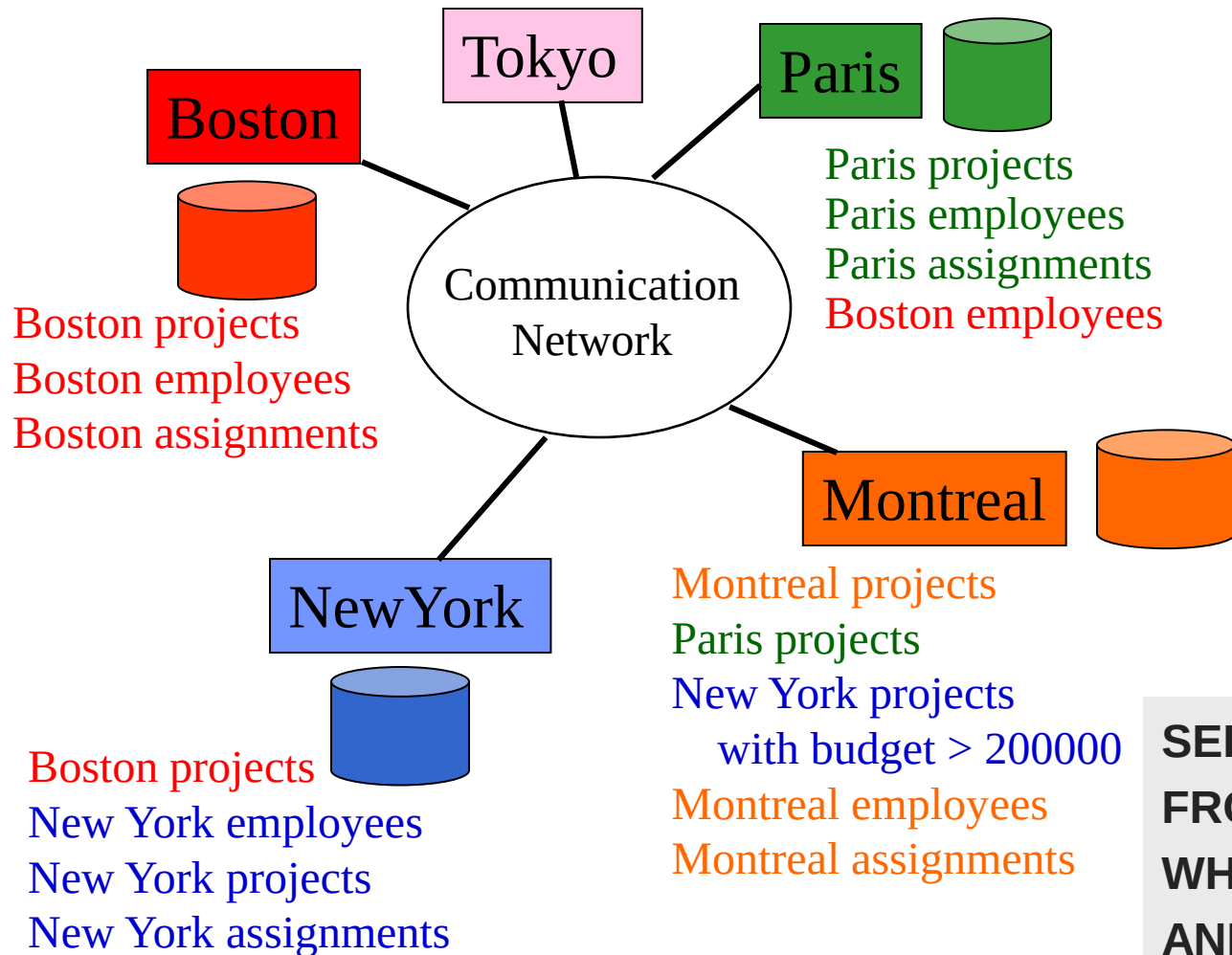
PROJ

PNO	PNAME	BUDGET
P1	Instrumentation	150000
P2	Database Develop	135000
P3	CAD/CAM	250000
P4	Maintenance	310000

PAY

TITLE	SAL
Elect. Eng.	40000
Syst. Anal.	34000
Mech. Eng.	27000
Programmer	24000

Transparent Access



Find the names and salaries of employees who are assigned to projects for over 12 weeks.

```
SELECT ENAME, SAL
FROM    EMP, ASG, PAY
WHERE   DUR > 12
AND     EMP.ENO = ASG.ENO
AND     PAY.TITLE = EMP.TITLE
```

Improved Performance

- Parallelism in execution
 - ◆ inter-query parallelism
 - ◆ intra-query parallelism
- Since each site handles only a portion of a database, the contention for CPU and I/O resources is not that severe.
- Data localization reduces communication overheads.
 - ◆ Proximity of data to its points of use requires some support from fragmentation and replication

Parallelism Requirements

- Have as much of the data required by *each* application at the site where the application executes
 - ◆ Full replication
- How about updates?
 - ◆ Updates to replicated data requires implementation of distributed **concurrency** control and **commit** protocol

Improved Reliability

- Distributed DBMS can use replicated components to eliminate single point failure.
- Users can still access part of the distributed database with “proper care” even though some of the data is unreachable.
- Distributed transactions facilitate maintenance of consistent database state even when failures occur.

Easier System Expansion

- Ability to add new sites, data, and users over time without major restructuring.
- Huge centralized database systems (mainframes) are history (almost!).
- Cloud computing will lead to natural distributed processing.
- Some applications (such as, supply chain, large enterprise) are naturally distributed - centralized systems will just not work.

Disadvantages of DDBSs

□ Complexity

- ◆ DDBS problems are inherently more complex than centralized DBMS ones

□ Cost

- ◆ More hardware, software, and people costs

□ Distribution of control

- ◆ Problems of synchronization and coordination to maintain data consistency

□ Security

- ◆ Database security + network security

□ Difficult to convert

- ◆ No tools to convert centralized DBMSs to DDBSs

Advantages of DDBSs

- 1. Increased reliability and availability** – A distributed database system is robust to failure to some extent. Hence, it is reliable when compared to a Centralized database system.
- 2. Local control** – The data is distributed in such a way that every portion of it is local to some sites (servers). The site in which the portion of data is stored is the owner of the data.
- 3. Modular growth (resilient)** – Growth is easier. We do not need to interrupt any of the functioning sites to introduce (add) a new site. Hence, the expansion of the whole system is easier. Removal of site is also does not cause much problems.
- 4. Lower communication costs (More Economical)** – Data are distributed in such a way that they are available near to the location where they are needed more. This reduces the communication cost much more compared to a centralized system.

Advantages of DDBSs

- 5. Faster response** – Most of the data are local and in close proximity to where they are needed. Hence, the requests can be answered quickly compared to a centralized system.
- 6. Reflects the organizational structure** – Normally, database is fragmented into various locations wherever we have controls.
- 7. Secured management of distributed data** – Various transparencies like network transparency, fragmentation transparency, and replication transparency are implemented to hide the actual implementation details of the whole distributed system. In such way, Distributed database provides security for data.
- 8. Robust** – The system is continued to work in case of failures. For example, replicated distributed database performs in spite of failure of other sites.
- 9. Complied with ACID properties** – Distributed transactions demands Atomicity, Consistency, Isolation, and Reliability.
- 10. Improved performance and Parallelism in executing transactions can be achieved.**

Dis-advantages of DDBSs

- 1. Complex Software** – Complex implementation. Costs more in terms of software cost compared to a centralized system. Additional software might be needed in most of the cases over a centralized system.
- 2. Increased Processing overhead** – It costs many messages to be shared between sites to complete a distributed transaction.
- 3. Data integrity** – Data integrity becomes complex. Too much network resources may be used.
- 4. Different data formats might be used** – This may cost time.
- 5. Deadlock is difficult to handle compared to a centralized system.**
- 6. May cause much more network traffic in case of write operation in a replicated form of distributed database.**
- 7. Distributed System supported Operating System is required to implement distributed database system.**
- 8. The data shared between sites over networks are vulnerable to attack.** Hence, network oriented security protocols to be used based on the sensitivity of data shared.
- 9. More complex in terms database design** – According to various applications, we may need to fragment a database, or replicate a database or both.
- 10. Handling failures is a difficult task.** In some cases, we may not distinguish site failure, network partition, and link failure.

Complicating Factors

- Data may be replicated in a distributed environment, consequently the DDBS is responsible for
 - ◆ choosing one of the stored copies of the requested data for access in case of retrievals
 - ◆ making sure that the effect of an update is reflected on each and every copy of that data item
- If there is site/link failure while an update is being executed, the DDBS must make sure that the effects will be reflected on the data residing at the failing or unreachable sites as soon as the system recovers from the failure.

Complicating Factors (*cont.*)

- Maintaining consistency of distributed/replicated data.
- Since each site cannot have instantaneous information on the actions currently carried out in other sites, the synchronization of transactions at multiple sites is harder than a centralized system.